Contents

1. ABI
    1. Type Layout
        1. Deviations from the Standard
        2. Fixed-Point Support
    2. Register Layout
        1. Fixed Registers
        2. Call-Used Registers
        3. Call-Saved Registers
    3. Frame Layout
    4. Calling Convention
        1. Exceptions to the Calling Convention
    5. Reduced Tiny
2. Extensions
    1. Types
    2. Attributes
    3. Pragmas
    4. Address Spaces
3. Using avr-gcc
    1. Supporting "unsupported" Devices
        1. avr-gcc v5 and newer
        2. avr-gcc v4.9 and below
4. Libf7
    1. Implementation
    2. Shortcomings
    3. Other Implementations

ABI

Application Binary Interface and implementation defined behaviour of avr-gcc. Object format bits are not discussed here. See also C Implementation-defined behaviour.

Type Layout

Endianess: Little

default	sizeof	Note
char	1	signed
short	2	
int	2	
long	4	
long long	8	
size_t	2	unsigned int
ptrdiff_t	2	int
void*	2	
float	4	
double	4,8	depends on configuration and command line options
long double	8,4	depends on configuration and command line options
wchar_t	2	

Deviations from the Standard

double

long double

In avr-gcc up to v9, `double` and `long double` are only 32 bits wide and implemented in the same way as `float`.

In avr-gcc v10 and higher, the layout of `double` and `long double` are determined by configure options `--with-double=` and `--with-long-double=`, respectively. The default layout of double is like float, and the default layout of long double is a 64-bit IEEE

format, see GCC configure options for details. Depending on the configuration, command line options `-mdouble=32` and `-mdouble=64` are available so that the type layout of `double` can be chosen at compile time, similar for `-mlong-double=32` and `-mlong-double=64` for `long double`. In order to test in a program which type layout has been chosen, GCC built-in macros `__SIZEOF_DOUBLE__` and `__SIZEOF_LONG_DOUBLE__` can be used.

8-bit int with -mint8

With `-mint8` int is only 8 bits wide which does not comply to the C standard. Notice that `-mint8` is not a multilib option and neither supported by AVR-Libc (except `stdint.h`) nor by newlib.

`-mint8`	sizeof	Note
char	1	signed
short	1	
int	1	
long	2	
long long	4	
size_t	2	long unsigned int
prtdiff_t	2	long int

Fixed-Point Support

avr-gcc 4.8 and up supports fixed point arithmetic according to ISO/IEC TR 18037. The support is not complete. The type layouts are as follows:

Type	sizeof	unsigned	signed	Note
`_Fract`				
short	1	0.8	±.7	
—	2	0.16	±.15	
long	4	0.32	±.31	
long long	8	0.64	±.63	GCC extension
`_Accum`				
short	2	8.8	±8.7	
—	4	16.16	±16.15	
long	8	32.32	±32.31	
long long	8	16.48	±16.47	GCC extension

Overflow behaviour of the non-saturated arithmetic is unspecified.

Please notice that some private ports found on the web implement different layouts.

Register Layout

Values that occupy more than one 8-bit register start in an even register.

Fixed Registers

Fixed Registers are registers that won't be allocated by GCC's register allocator. Registers R0 and R1 are fixed and used implicitly while printing out assembler instructions:

R0

is used as scratch register that need not to be restored after its usage. It must be saved and restored in interrupt service routine's (ISR) prologue and epilogue. In inline assembler you can use `__tmp_reg__` for the scratch register.

R1

always contains zero. During an insn the content might be destroyed, e.g. by a MUL instruction that uses R0/R1 as implicit output register. If an insn destroys R1, the insn must restore R1 to zero afterwards. This register must be saved in ISR prologues and must then be set to zero because R1 might contain values other than zero. The ISR epilogue restores the value. In inline assembler you can use `__zero_reg__` for the zero register.

T

the T flag in the status register (SREG) is used in the same way like the temporary scratch register R0.

User-defined global registers by means of global `register asm` and / or `-ffixed-`*n* won't be saved or restored in function pro- and epilogue.

Call-Used Registers

The *call-used* or *call-clobbered* general purpose registers (GPRs) are registers that might be destroyed (clobbered) by a function call.

R18–R27, R30, R31
: These GPRs are call clobbered. An ordinary function may use them without restoring the contents. Interrupt service routines (ISRs) must save and restore each register they use.

R0, T-Flag
: The temporary register and the T-flag in SREG are also call-clobbered, but this knowledge is not exposed explicitly to the compiler (R0 is a fixed register).

Call-Saved Registers

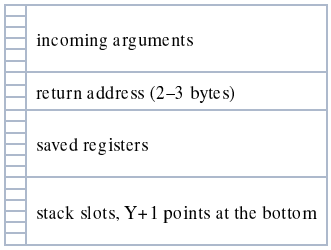
R2–R17, R28, R29
: The remaining GPRs are call-saved, i.e. a function that uses such a registers must restore its original content. This applies even if the register is used to pass a function argument.

R1
: The zero-register is implicity call-saved (implicit because R1 is a fixed register).

Frame Layout

During compilation the compiler may come up with an arbitrary number of *pseudo registers* which will be allocated to *hard registers* during register allocation.

Frame Layout after Function Prologue



- Pseudos that don't get a hard register will be put into a stack slot and loaded / stored as needed.
- In order to access stack locations, avr-gcc will set up a 16-bit frame pointer in R29:R28 (Y) because the stack pointer (SP) cannot be used to access stack slots.
- The stack grows downwards. Smaller addresses are at the bottom of the drawing at the right.
- Stack pointer and frame pointer are not aligned, i.e. 1-byte aligned.
- After the function prologue, the frame pointer will point one byte below the stack frame, i.e. Y+1 points to the bottom of the stack frame.
- Any of "incoming arguments", "saved registers" or "stack slots" in the drawing at the right may be empty.
- Even "return address" may be empty which happens for functions that are tail-called.

Calling Convention

- An argument is passed either completely in registers or completely in memory.
- To find the register where a function argument is passed, initialize the register number R_n with R26 and follow this procedure:
  1. If the argument size is an odd number of bytes, round up the size to the next even number.
  2. Subtract the rounded size from the register number R_n.
  3. If the new R_n is at least R8 and the size of the object is non-zero, then the low-byte of the argument is passed in R_n. Subsequent bytes of the argument are passed in the subsequent registers, i.e. in increasing register numbers.
  4. If the new register number R_n is smaller than R8 or the size of the argument is zero, the argument will be passed in memory.
  5. If the current argument is passed in memory, stop the procedure: All subsequent arguments will also be passed in memory.
  6. If there are arguments left, goto 1. and proceed with the next argument.
- Return values with a size of 1 byte up to and including a size of 8 bytes will be returned in registers. Return values whose size is outside that range will be returned in memory.
- If a return value cannot be returned in registers, the caller will allocate stack space and pass the address as implicit first pointer argument to the callee. The callee will put the return value into the space provided by the caller.
- If the return value of a function is returned in registers, the same registers are used as if the value was the first parameter of a non-varargs function. For example, an 8-bit value is returned in R24 and an 32-bit value is returned R22...R25.
- Arguments of varargs functions are passed on the stack. This applies even to the named arguments.

For example, suppose a function with the following prototype:

```
int func (char a, long b);
```

then

- a will be passed in R24.
- b will be passed in R20, R21, R22 and R23 with the LSB in R20 and the MSB in R23.
- The result is returned in R24 (LSB) and R25 (MSB).

Exceptions to the Calling Convention

GCC comes with libgcc, a runtime support library. This library implements functions that are too complicated to be emit inline by GCC. What functions are used when depends on the target architecture, what instructions are available, how expensive they are and on the optimization level.

Functions in libgcc are implemented in C or hand-written assembly. In the latter case, some functions use a special ABI that allows better code generation by the compiler.

For example, the function that computes unsigned 8-bit quotient and remainder, `__udivmodqi4`, just returns the quotient and the remainder and clobbers R22 and R23. The compiler knows that the function does not destroy R30, for example, and may hold a value in R30 across the function call. This reduces the register pressure in functions that call `__udivmodqi4`.

Function	Availability	Operation	Clobbers	Description
`__umulhisi3`	4.7+ && MUL	`SI:22 = HI:26 * HI:18`	R_{tmp}	Multiply 2 unsigned 16-bit integers to a 32-bit result
`__mulhisi3`	4.7+ && MUL	`SI:22 = HI:26 * HI:18`	R_{tmp}	Multiply 2 signed 16-bit integers to a 32-bit result
`__usmulhisi3`	4.7+ && MUL	`SI:22 = HI:26 * HI:18`	R_{tmp}	Multiply the signed 16-bit integer in R26 with the unsigned 16-bit integer in R18 to a 32-bit result
`__muluhisi3`	4.7+ && MUL	`SI:22 = HI:26 * SI:18`	R_{tmp}	Multiply an unsigned 16-bit integer with a 32-bit integer to a 32-bit result
`__mulshisi3`	4.7+ && MUL	`SI:22 = HI:26 * SI:18`	R_{tmp}	Multiply a signed 16-bit integer with a 32-bit integer to a 32-bit result
`__udivmodqi4`		`QI:24 = QI:24 / QI:22` `QI:25 = QI:24 % QI:22`	R_{23}	Unsigned 8-bit integer quotient and remainder
`__divmodqi4`		`QI:24 = QI:24 / QI:22` `QI:25 = QI:24 % QI:22`	R_{23}, R_{tmp}, T	Signed 8-bit integer quotient and remainder
`__udivmodhi4`		`HI:22 = HI:24 / HI:22` `HI:24 = HI:24 % HI:22`	R_{21}, $R_{26...27}$	Unsigned 16-bit integer quotient and remainder
`__divmodhi4`		`HI:22 = HI:24 / HI:22` `HI:24 = HI:24 % HI:22`	R_{21}, $R_{26...27}$, R_{tmp}, T	Signed 16-bit integer quotient and remainder

The *Operation* column uses GCC's machine modes to describe how values in registers are interpreted.

Machine Modes	**Q**arter, 8 bit	**H**alf, 16 bit	**S**ingle, 32 bit	**D**ouble, 64 bit	**P**artial **S**ingle, 24 bit
Integer	QI	HI	SI	DI	PSI
Float	–	–	SF	DF	–
Signed _**A**ccum	–	HA	SA	DA	–
Signed _Fract (**Q**-Format)	QQ	HQ	SQ	DQ	–
Unsigned _**A**ccum	–	UHA	USA	UDA	–
Unsigned _Fract (**Q**-Format)	UQQ	UHQ	USQ	UDQ	–

Reduced Tiny

On the Reduced Tiny cores (16 GPRs only) several modifications to the ABI above apply:

- Call-saved registers are: R18–R19, R28–R29.
- Fixed Registers are R16 (`__tmp_reg__`) and R17 (`__zero_reg__`).
- Registers used to pass arguments to functions and return values from functions are R25...R18 (instead of R25...R8).

There is only limited library support both from libgcc and AVR-LibC, for example there is no float support and no printf support.

Extensions

Types

- Signed and unsigned 24-bit integers: `__int24` (v4.7), `__uint24` (v4.7).

Attributes

- Variable: `progmem`, `absdata` (v7).
- Function: `interrupt`, `signal`, `naked`, `OS_main` (v4.4), `OS_task` (v4.4), `no_gccisr` (v8).
- Type: (none).

Pragmas

(none)

Address Spaces

`__flash` (v4.7), `__flash1` ... `__flash5` (v4.7), `__memx` (v4.7).

Using avr-gcc

Supporting "unsupported" Devices

avr-gcc v5 and newer

avr-gcc v4.9 and below

avr-gcc and avr-as support the `-mmcu=`*device* command line option to generate code for a specific device. Currently (2012), there are more than 200 known AVR devices and the hardware vendor keeps releasing new devices. If you need support for such a device and don't want to rebuild the tools, you can

1. Sit and wait until support for your `-mmcu=`*device* is added to the tools.
2. Use appropriate command line options to compile for your favourite *device*.

Approach 1 is comfortable but slow. Lazy developers that don't care for time-to-market will use it.

Approach 2 is preferred if you want to start development as soon as possible and don't want to wait until the tool chain with respective device support is released. **This approach is only possible if the compiler and Binutils already come with support for the core architecture of your device.**

When you feed code into the compiler and compile for a specific device, the compiler will only care for the respective core; it won't care for the exact device. It does not matter to the compiler how many I/O pins the device has, at what voltage it operates, how much RAM is present, how many timers or UARTs are on the silicon or in what package it is shipped. The only thing the compiler does with `-mmcu=`*device* is to build-in define a specific macro and to call the linker in a specific way, i.e. the compiler driver behaves a bit differently, but the sub-tools like compiler proper and assembler will generate exactly the same code.

Thus, you can support your device by setting these options by hand.

Additionally, we need the following to compile a C program:

- A device support header `avr/io.h` similar to the headers provided by AVR Libc.
- Startup code for the device.

The Device Header avr/io.h

This header and its subheaders contain almost all information about a particular device like SFR addresses, size of the interrupt table and interrupt names, etc.

After all, it's just text and you can write it yourself. Find a device that is already supported by AVR-Libc and that is similar enough to your new device to serve as a reasonable starting point for the new device description.

If you are lucky, the device it already supported by AVR-Libc but not yet by the compiler. In that case, you can use verbatim copies from AVR-Libc.

Yet another approach is to write the file from scratch or not to use `avr/io.h` like headers at all. I that case, you provide all needed definitions like, say, SP and size of the vector table yourself.

If your toolchain is distributed with AVR-Libc then `avr/io.h` is located in the installation directory at `./avr/include` i.e. you find a file `io.h` in `./avr/include/avr`. In that file you find the lines:

```
#if defined (__AVR_AT94K__)
#  include <avr/ioat94k.h>
#elif defined (__AVR_AT43USB320__)
#  include <avr/io43u32x.h>

/* many many more entries */

#else
#  if !defined(__COMPILING_AVR_LIBC__)
#    warning "device type not defined"
#  endif
#endif
```

Add an entry for `__AVR_mydevice__` and include your new file `avr/iomydevice.h`.

If you don't want to change the existing `avr/io.h` then copy it to a new directory and add that directory as system search path by means of `-isystem` whenever you compile or preprocess a C or assembler source that shall include the extended `avr/io.h`. Notice that the new directory will contain a subdirectory named `avr`.

Compiling the Code

Let's start with a simple C program, `source.c`:

```
#include <avr/io.h>

int var;

int main (void)
{
    return var + SP;
}
```

Your source directory then contains the following files:

```
source.c    gcrt1.S    macros.inc    sectionname.h
```

The startup code gcrt1.S and macros.inc are verbatim copies from AVR-Libc.

`sectionname.h` is included by `macros.inc` but we don't need it: Simply provide `sectionname.h` as an empty file.

For the matter of simplicity, we show how to compile for a device that is similar to ATmega8 so that we don't need to extend `avr/io.h` to show the work flow. In the case you copied `avr/io.h` to a new place, don't forget to add respective `-isystem` to the first two commands for `source.c` and `gcrt1.S`.

ATmega8 is a device in core family `avr4`, thus we compile and assemble our `source.c` for that core architecture. `__AVR_ATmega8__` stands for the subheader selector you added to `avr/io.h`.

```
avr-gcc -mmcu=avr4 -D__AVR_ATmega8__ -c source.c -Os
```

Similarly, we assemble the startup code for our device by means of:

```
avr-gcc -mmcu=avr4 -D__AVR_ATmega8__ -c gcrt1.S -o crt0-mydevice.o
```

Finally, we link the stuff together to get a working `source.elf` (assuming that RAM starts at address 0x124):

```
avr-gcc -mmcu=avr4 -Tdata 0x800124 source.o crt0-mydevice.o -nostartfiles -o source.elf
```

Voilà!

Libf7

Libf7 is an ad-hoc, AVR-specific, 64-bit floating point emulation written in GNU-C and (inline) assembly. It is hosted and deployed as part of libgcc. Hence, it will be part of any avr-gcc distribution from v10 onwards without any further ado, except the compiler is configured with `--with-libf7=no`.

Implementation

- The emulated 64-bit floating point representation is IEEE compatible: Little endian, 11 bit for the encoded exponent, 52 bits for the encoded mantissa.
- The internal, decoded representation uses `f7_t`, a struct with 1 byte for flags, 7 bytes for the mantissa and 2 bytes for the exponent.
- Functions like `sin` are wrappers that convert `double` (or `long double` for that matter) to `f7_t` and then call functions prefixed with `__f7_` that operate on the internal representation. In the case of `sin`, the worker function has prototype
  ```
  void __f7_sin (f7_t*, const f7_t*)
  ```
- The basic arithmetic is implemented mostly in assembly. Functions like `__f7_sin` are implemented in C.
- The transcendental functions are implemented using MiniMax approximations, i.e. they minimize the maximum norm. Most of these functions use rational MiniMax approximations because they perform better than CORDIC (and they perform better than Taylor or Padé expansions, of course). Square-root uses 3 iterations of Newton-Raphson.
- Portability to other architectures or to other compilers was of no consideration; the implementation focuses solely on avr-gcc. This means that if you want to implement a 64-bit floating point emulation to be used elsewhere, Libf7 is of no use — except that the used algorithms and MiniMax polynomials might provide you some additional perspectives.

Shortcomings

Libf7 is incomplete:

- Assembly routines that would require MUL instructions are not implemented. This means that when you try to link programs with 64-bit double for a device without MUL, you will get an undefined reference from the linker like for `__f7_mul_mant_asm`.
- Some functions from math.h like `atan2`, `lround`, `lrint`, `fma`, Bessel and Gamma are not implemented. If you try to use them, you will get undefined references from the linker; in the case of `atan2` the missing function is `__f7_atan2`. If you really need it, you can provide such functions in your projects, or better still contribute them to GCC.
- Signed zeroes are handled like zero.

Libf7 is not optimal:

- `sqrt` uses 3 steps of Newton-Raphson iteration. Using an old-school algorithm might reduce code size (after considering all dependencies). And it would be faster, of course, speeding up `sqrt`, `hypot`, `asin`, `acos` and their `long double` variants.

Other Implementations

- fp64lib from *Uwe Bissinger*: Written in GNU assembly. Slightly less precise. Roughly the same speed (except for square root which is several times faster). Smaller stack footprint. Slightly smaller code for basic arithmetic, otherwise comparable code sizes. No build script / Makefile as it targets the Arduino ecosystem.
- avr_64.c from *Detlef* with improvements from *Florian Königstein*: Implemented in C. Resource consumption might be a multiple of what Libf7 consumes. Easy to integrate in own projects that use avr-gcc without native 64-bit double support. Precision is quite good except for some corner(?) cases where it might deteriorate. Could be compiled after fixing minor problems (missing `const` at `progmem`). Should also work with other compilers / targets.
- dannis64bit.S from *Peter Danegger*. Written in GNU assembly.

None: avr-gcc (last edited 2020-01-14 16:56:04 by GJLay)